

#중급반 6주차

STL 심화

<Advanced STL>

3p /<std::map>

10p /<std::set>

17p /<std::priority_queue>

22p /<연습 문제>



#STL

<C++ Standard Library>

C++의 **Standard Library**를 통해 미리 구현된 여러 **자료구조**들을 사용할 수 있습니다.

vector, string, stack, queue 등이 STL에 포함된 자료구조입니다.

map, set, priority_queue 에 대해 알아보시다!



#STL

<C++ Standard Library> - map

map은 key와 value의 쌍을 저장하는 자료구조입니다.

다음 특징을 가지고 있습니다.

- key는 중복되지 않으며, (기본적으로는) 오름차순으로 정렬되어 있다.
- key를 통해서 value를 시간복잡도 $O(\log N)$ 만에 찾을 수 있다.
- key-value 쌍의 삽입과 삭제 연산 또한 $O(\log N)$ 이다.

#STL

<C++ Standard Library> - map

1. map 만들기

map을 만들기 위해서는 먼저,
<map>이라는 헤더를 불러와야 합니다.
이후 map<key 자료형, value 자료형> 형태로
key와 value의 자료형을 지정합니다.

greater를 이용해 key가 반대로
정렬되게 할 수도 있습니다.

```
#include <iostream>
#include <string>
#include <map>

// map 또한 std 안에 들어있습니다.
using namespace std;

int main() {
    map<string, int> m1;

    // 내림차순
    map<string, bool, greater<string>> m2;
}
```

#STL

<C++ Standard Library> - map

2. 삽입과 삭제 및 읽기 연산

map을 활용하는 주된 연산들을 알아보시다.

```
int main() {  
    map<string, int> m;  
  
    m["A"] = 10; // 삽입  
  
    m["A"] = 20; // 수정(이미 존재할 경우)  
  
    cout << m["A"]; // 읽기, 출력: 20  
  
    // 없는 key는 기본값이 삽입됨, 출력: 0  
    cout << m["B"];  
  
    m.erase("A"); // 삭제  
}
```

#STL

<C++ Standard Library> - map

3. map 전체 순회하기

map의 내용물을 전부 확인하는
여러 가지 방법을 알아봅시다.

```
int main() {  
    map<string, int> m;  
    m["A"] = 10; m["B"] = 20; m["C"] = 30;  
  
    // 1. 범위 기반 for 문 (std::pair 참조)  
    for (auto &p : m) { // p: pair<string, int>  
        cout << p.first << ' ' << p.second << '\n';  
    }  
    cout << "-----\n";  
  
    // 2. 구조화된 바인딩 (C++17 이상)  
    for (auto &[key, value] : m) {  
        cout << key << ' ' << value << '\n';  
    }  
}
```

#STL

<C++ Standard Library> - map

3. map 전체 순회하기

map의 내용물을 전부 확인하는
여러 가지 방법을 알아봅시다.

```
int main() {  
    map<string, int> m;  
    m["A"] = 10; m["B"] = 20; m["C"] = 30;  
  
    // 3. Iterator 타입을 명시적으로 선언하여 순회  
    for (map<string, int>::iterator it = m.begin();  
         it != m.end(); ++it) {  
        // 반복자는 포인터처럼 동작하므로  
        // -> 연산자를 사용해 접근합니다.  
        cout << it->first << ' ' << it->second << '\n';  
    }  
    cout << "-----\n";  
  
    // 4. auto를 사용하여 반복자 선언을 간소화한 순회  
    for (auto it = m.begin(); it != m.end(); ++it) {  
        cout << it->first << ' ' << it->second << '\n';  
    }  
}
```



#STL

<C++ Standard Library>

Tutorial : STL Map 1 (기본 개념) (JUNGOL #8561)

<문제 설명>

- 사람들의 이름이 영어 소문자로 이뤄진 문자열로 주어짐
- 각 이름들이 몇 번째 줄에서 마지막으로 입력됐는지,
사전 순으로 정렬하여 출력

<제약 조건>

- (이름의 길이) ≤ 50
- (단어의 수) $\leq 100,000$

#STL

<C++ Standard Library>

4 Tutorial : STL Map 1 (기본 개념) (JUNGOL #8561)

<문제 해설>

```
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    map<string, int> name_map;
    string name;
    int line_number = 0;

    while(true) {
        cin >> name
        if (name == "end") {
            break;
        }
    }
```

```
        line_number++;
        name_map[name] = line_number;
    }

    cout << name_map.size() << "\n";

    for (const auto& pair : name_map) {
        cout << pair.first << " ";
        cout << pair.second << "\n";
    }

    return 0;
```

}



#STL

<C++ Standard Library> - set

set은 요소를 **중복 없이 저장하는** 자료구조입니다.

다음 특징을 가지고 있습니다.

- 삽입과 삭제, 찾기 연산이 모두 $O(\log N)$ 에 동작한다.
 - 항상 요소는 **정렬된 순서를 유지**한다.
- 등이 있습니다.

#STL

<C++ Standard Library> - set

1. set 만들기

set을 만들기 위해서는 먼저,

`<set>`이라는 헤더를 불러와야 합니다.

이후 `set<자료형>` 형태로 타입을 지정합니다.

`greater`를 이용해

반대로 정렬되게 할 수도 있습니다.

```
#include <iostream>
#include <string>
#include <set>

using namespace std;

int main() {
    set<string> s1;

    // 내림차순
    set<int, greater<int>> s2;
}
```

#STL

<C++ Standard Library> - set

2. 요소 삽입과 삭제 연산

set에 이미 있는 요소를 삽입하거나
set에 없는 요소를 삭제하는 경우,
아무 일도 일어나지 않습니다.

```
int main() {  
    set<int> s;  
  
    s.insert(0);  
    s.insert(1);  
  
    // 아무 일도 일어나지 않음  
    s.insert(1);  
  
    s.erase(1);  
  
    // 아무 일도 일어나지 않음  
    s.erase(2);  
}
```

#STL

<C++ Standard Library> - set

3. 요소 찾기

find를 이용해 어떤 요소가 set에
존재하는지 아닌지를 알 수 있습니다.

find의 반환값은 요소가 있는 위치를 가리키는 이터레이터,
혹은 없을 경우 end를 가리키는 이터레이터를 반환합니다.
'이터레이터'는 당장 설명하기에는 어렵기 때문에,
이번 멘토링에서는 다루지 않습니다.

```
int main() {  
    set<int> s;  
  
    s.insert(0); s.insert(1); s.insert(2);  
  
    // find는 요소가 없는 경우에만  
    // end와 같은 결과를 반환합니다.  
    cout << (s.find(1) == s.end()); // 0 (false)  
    cout << (s.find(3) == s.end()); // 1 (true)  
  
    // 버전이 C++20 이상이라면  
    // 아래 코드를 사용할 수 있습니다.  
    cout << s.contains(2); // 1 (true)  
}
```

#STL

<C++ Standard Library> - set

4. 모든 요소 순회

set에 존재하는 모든 요소를 탐색해봅시다.

```
int main() {  
    set<int> s;  
    s.insert(2); s.insert(1); s.insert(0);  
  
    // 1. 범위 기반 for 문  
    for (auto &item: s) {  
        cout << item << ' ';  
    } cout << "\n";  
  
    // 2. 반복자(Iterator) 타입을 명시적으로 선언하여 순회  
    for (set<int>::iterator it = s.begin();  
        it != s.end(); ++it) {  
        // set의 반복자는 값을 바로 가리키므로  
        // 포인터처럼 * 연산자를 붙여서 출력합니다.  
        cout << *it << ' ';  
    } cout << "\n";  
  
    // 3. auto를 사용하여 반복자 선언을 간소화한 순회  
    for (auto it = s.begin(); it != s.end(); ++it) {  
        cout << *it << ' ';  
    }  
    cout << "\n";  
}
```



#STL

<C++ Standard Library>

 Tutorial: STL Set 1 (JUNGOL #4637)

<문제 설명>

- 쿼리가 Q개 주어짐
 - i x : x가 없다면 집합에 추가
 - r x : 집합 내에 x가 있으면 삭제, 없다면 아무 것도 하지 않음

<제약 조건>

- $1 \leq Q \leq 20,000$
- $-10^9 \leq x \leq 10^9$



#STL

<C++ Standard Library>

Tutorial: STL Set 1 (JUNGOL #4637)

<문제 해설>

```
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int q;
    cin >> q;
    set<int> s;
    for (int i = 0; i < q; ++i) {
        char op; int num;
        cin >> op >> num;
        if (op == 'i') s.insert(num);
        else if (op == 'r') s.erase(num);
    }
    // set은 내부적으로 항상 오름차순
    for (int x : s) cout << x << " ";
    cout << "\n";
}
```




#STL

<C++ Standard Library> - priority_queue

priority_queue는 항상 가장 큰/작은 요소를 꺼낼 수 있게 해주는 자료구조입니다.

다음 특징을 가지고 있습니다.

- 요소의 삽입과 삭제 연산이 모두 $O(\log N)$ 에 동작한다.
- top에는 항상 가장 큰/작은 요소가 들어있다.

#STL

<C++ Standard Library> - priority_queue

1. priority_queue 만들기

priority_queue를 만들기 위해서는 먼저,
<queue> (+ <vector>)라는 헤더를 불러와야
합니다.

이후, priority_queue<자료형>으로 타입을 정
합니다.

```
#include <vector>
#include <queue>

using namespace std;

int main() {
    // 가장 큰 게 우선
    priority_queue<int> pq1;

    // 가장 작은 게 우선:
    // <타입, 저장할 공간, 비교 함수> 순서
    priority_queue<int, vector<int>, greater<int>> pq2;
}
```

#STL

<C++ Standard Library> - priority_queue

2. 삽입과 삭제 및 읽기 연산

priority_queue의 주된 연산들을 알아보시다.

```
#include <iostream>
#include <queue>
#include <vector>

using namespace std;

int main() {
    priority_queue<int> pq;
    pq.push(10); pq.push(30);

    cout << pq.top() << '\n'; // 30
    cout << pq.size() << '\n'; // 2
    cout << pq.empty() << '\n'; // 0 (false)

    pq.pop(); // 반환값 없이 가장 큰 요소만 삭제
}
```



#STL

<C++ Standard Library>

 1 하늘에서 떨어지는 돈 (JUNGOL #8589)

<문제 설명>

- N명의 사람이 가진 초기 자금이 주어짐
- 총 M번에 걸쳐 돈이 떨어질 때마다,
현재 가장 돈이 적은 사람이 그 돈을 가져감
- 모든 과정이 끝난 후, 사람들이 가진 최종 금액을
오름차순으로 정렬하여 출력

<제약 조건>

- $1 \leq N \leq 200,000$
- $0 \leq a_i \leq 10^9$
- $1 \leq M \leq 200,000$
- $0 \leq (\text{떨어지는 돈}) \leq 10^9$

#STL

<C++ Standard Library>

1 하늘에서 떨어지는 돈 (JUNGOL #8589)

```
typedef long long ll;
int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    int n;
    cin >> n;
    priority_queue<ll, vector<ll>, greater<ll>> pq;

    for (int i = 0; i < n; ++i) {
        ll money;
        cin >> money;
        pq.push(money);
    }

    int m;
    cin >> m;
```

```
    for (int i = 0; i < m; ++i) {
        ll dropped_money;
        cin >> dropped_money;

        // 현재 가장 돈이 적은 사람의 금액을 가져옴
        ll least_money = pq.top();
        pq.pop();

        // 떨어진 돈을 더해서 다시 힙에 삽입
        pq.push(least_money + dropped_money);
    }

    // 최소 힙에서 하나씩 꺼내면 자동으로 오름차순 정렬
    while (!pq.empty()) {
        cout << pq.top() << " ";
        pq.pop();
    }
}
```



#필수 문제

<C++ Standard Library>

4 Tutorial : STL Map 1 (기본 개념) (JUNGOL #8561)

std::map에 익숙해지기

2 Tutorial: STL Set 1 (JUNGOL #4637)

std::set에 익숙해지기

1 하늘에서 떨어지는 돈 (JUNGOL #8589)

우선순위 큐의 중요한 성질을 알아보기

#연습 문제

<C++ Standard Library>

2 Tutorial : STL Priority Queue (JUNGOL #8587)

연산자 오버로딩에 대해서도 알아 볼까요?

3 평행하는 선분의 수 (JUNGOL #8036)

오늘 배운 내용을 어떻게 활용할 수 있을까요?

4 별의 대칭 (JUNGOL #11048)

2023 ICPC Seoul Regional 예선 K번

4 Tutorial : STL Map 2 (원소 검색) (JUNGOL #8562)

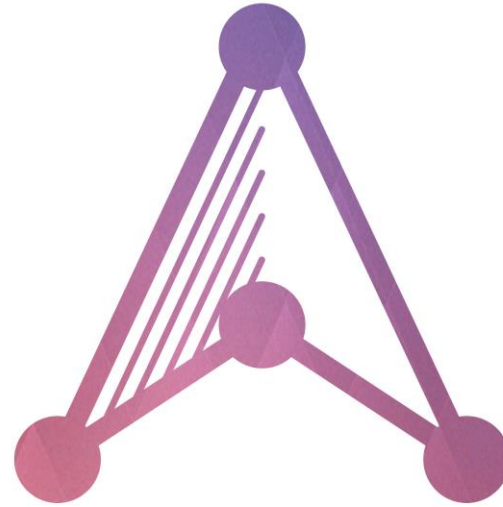
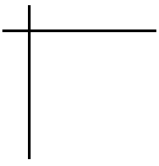
std::map에 조금 더 익숙해지기

5 나는 기말고사형 인간이야 (JUNGOL #8278)

나도 기말고사형 인간인 거 같은데

3 중앙값 (JUNGOL #1570)

코테에도 많이 나오는 아이디어



A L O H A
The algorithm club.

